UNIVERSIDAD DE LAS FUERZAS ARMADAS – ESPE
Department of Computer Science
Software Engineering
36862 – Object-Oriented Programming

UNIT 1 PROJECT · TEAM WORK

UML Design Specification: ProvStock

Use Case Diagram and Class Diagram

Team	ByteWorks
Members	Andrés Pachacama, Carlos Parra, Matias Paucar
NRC	36862
Instructor	Edison Lascano, PhD
Date	October 26, 2026
Academic period	SII-2026 (September 2026 – February 2027)
Version	1.3

Revision History

Table 1

Document Versions

Version	Date	Authors	Description of Changes
1.0	October 20, 2026	Team ByteWorks	Use case diagram aligned with the SRS v1.2 (20 use cases) and class diagram (22 classes), which replace the diagrams of homework 06 and 07.
1.1	October 21, 2026	Team ByteWorks	Class diagram aligned with the Java implementation: the «use» dependencies between control classes become directed associations with their attribute, the methods findUser(), findRole(), and findByRuc() are added, the dependency AccessManager «use» StockMonitor is removed, and notes N8 to N10, the table of permissions per role (Table 10), and new method rules are added. The details are in Section 5.2.
1.2	October 25, 2026	Team ByteWorks	Data persistence: new class FileManager (Diagram C, Figure 5) that saves the data in CSV files and loads it again at startup, restore...() methods in the control classes and in Sale and Purchase, updated Figures 2 and 4, notes N11 and N12, and new rules. The details are in Section 5.3.
1.3	October 26, 2026	Team ByteWorks	Data saved in JSON files, one per class, with the Gson library; reports exported to CSV by the new class CsvReportExporter (Export report to CSV is now included by Generate reports); clean code changes: new classes ProductInfo and SaleFilter, `find...()` methods that throw exceptions instead of returning null, and `register...()` methods that return void; updated Figures 1 to 5, notes N13 and N14, and new rules. The section of planned future changes was removed. The details are in Section 5.4.

Table of Contents

Revision History 2
1. Introduction 4
    1.1 Purpose 4
    1.2 Scope of the Class Diagram 5
    1.3 Conventions 5
2. Use Case Diagram 7
3. Class Diagram 11
    3.1 Overview 11
    3.2 Domain Classes (Diagram A) 18
    3.3 Control Classes (Diagram B) 22
    3.4 Persistence Classes (Diagram C) 24
    3.5 Relationships and Multiplicities 26
    3.6 Design Notes 29
    3.7 Rules Implemented by the Methods 31
4. Traceability 34
5. Changes from Previous Versions 35
    5.1 Changes from the Diagram of Homework 07 35
    5.2 Changes in Version 1.1 35
    5.3 Changes in Version 1.2 36
    5.4 Changes in Version 1.3 37

1. Introduction

1.1 Purpose

This document specifies the use case diagram and the class diagram of ProvStock so that they can be drawn in Visual Paradigm and serve as the basis for the Java implementation. It replaces the diagrams delivered in homework 06 and 07 and is consistent with the Software Requirements Specification (SRS) v1.3 (Pachacama et al., 2026a) and with the Use Case Specifications v1.2 (Pachacama et al., 2026b).

Version 1.1 corrects the class diagram so that it matches the Java code that already implements the system. During programming, the team found details that the diagram did not solve; for example, where StockMonitor gets the list of products if it has no attributes and receives no parameters. These adjustments are explained in notes N8 to N10 and summarized in Section 5.2.

Version 1.2 adds data persistence: the new class `FileManager` saves the information in CSV files and loads it again when the program opens, so nothing is lost when it is closed (Section 3.4). The changes are summarized in Section 5.3. It also updates the use case diagram: the «extend» of Manage customers and the «include» relationships that described the internal behavior of the system were removed, and Receive stock alerts now includes Check minimum stock.

Version 1.3 applies the changes requested by the instructor and a review of the code based on the book Clean Code (Martin, 2008): the data is saved in JSON files, one per class, with the Gson library (Section 3.4); the reports are exported to CSV files by the new class `CsvReportExporter`; and the new classes `ProductInfo` and `SaleFilter` group parameters that used to be passed separately. The `find...()` methods now throw an exception when the object

does not exist, and the `register...()` methods return void (notes N13 and N14). The changes are summarized in Section 5.4.

The names of classes, attributes, methods, and data types follow the Java naming conventions (NFR-05), because they are part of the code. When a class is mentioned, its meaning is given; for example, the sale (`Sale`) or the stock movement (`StockMovement`).

1.2 Scope of the Class Diagram

The class diagram uses the concepts covered so far in the course: classes, attributes, methods, associations, composition, aggregation, dependencies, and multiplicities. The classes that share data use the same attribute and method names (note N3), so they are used in the same way.

1.3 Conventions

Table 2 summarizes the UML notation used in the class diagram.

Table 2

Notation Used in the Class Diagram

Element	Meaning
- / +	Private / public visibility. A private attribute can be used only inside its own class (encapsulation); a public method can be called from other classes.
`LocalDate`, `LocalDateTime`	Date types of the package `java.time` (date, and date with time); they replace the old class `java.util.Date`, which is deprecated.
`double`	Amounts of money and quantities (see note N1).
`String` with fixed values	Attributes such as `status` or `type` accept only the values listed in Section 3.6 (see notes N2 and N9).
Getters and setters	The simple methods to read (`get`) or change (`set`) an attribute are not drawn (note N6).
Association	Solid line between two classes indicating that their objects are related; it has a name and the multiplicity at each end.
Directed association (→)	Association with an open arrow at one end: the source class keeps a reference to the target class as an attribute and can use it, but not the other way around (note N8).

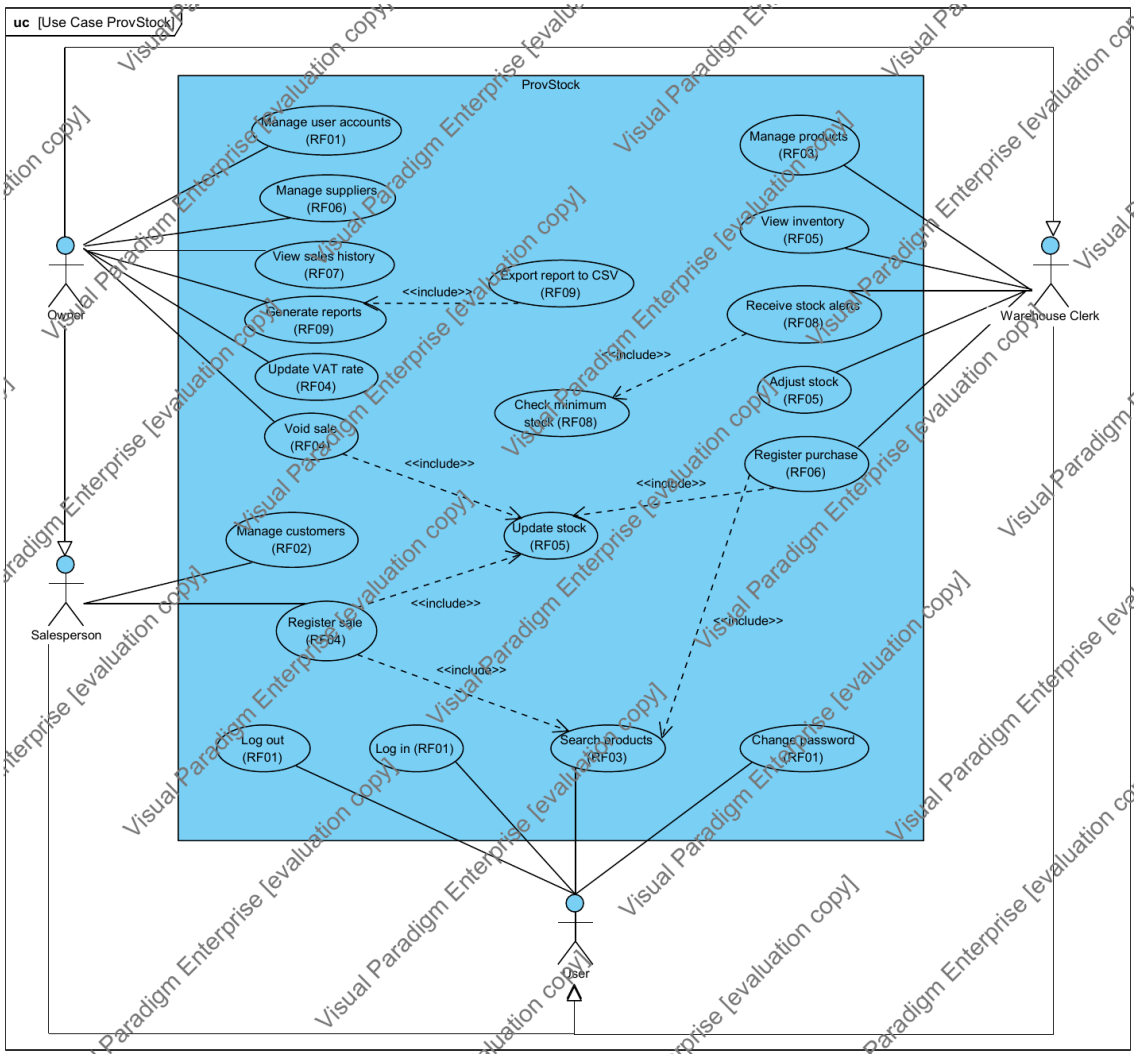
Element	Meaning
Multiplicity	How many objects of one class are related to one object of the other: `1` (exactly one), `0..1` (zero or one), `0..*` (zero or many), `1..*` (one or many).
Composition (◆)	Strong “whole-part” relationship: the part cannot exist without the whole (a sale line does not exist without its sale).
Aggregation (◇)	Weak “whole-part” relationship: a control class keeps a collection of objects that can exist on their own. If aggregation has not been covered yet, it can be drawn as a simple association with the same multiplicities.
Dependency «use» / «create»	Dashed arrow: a class uses or creates objects of another class without keeping them as an attribute.

2. Use Case Diagram

Figure 1 shows the use case diagram, identical to Figure 1 of the SRS v1.3. A use case represents something that an actor wants to achieve with the system; an actor is a role that interacts with the system. Tables 3 to 5 list every element so that it can be redrawn exactly in Visual Paradigm.

Figure 1

Use Case Diagram of ProvStock



Note. The abstract actor User represents any person with an account. Hollow arrows indicate actor generalization; the Salesperson and the Warehouse Clerk share one generalization toward User. Dashed arrows indicate «include» relationships. The codes in parentheses refer to the Product Backlog items.

Table 3

Actors and Generalizations

Actor	Generalization	Meaning
User (abstract)	—	Any person with an account.
Salesperson	Salesperson → User	Inherits the use cases of User.
Warehouse Clerk	Warehouse Clerk → User	Inherits the use cases of User.
Owner	Owner → Salesperson; Owner → Warehouse Clerk	Inherits all the use cases of both roles.

Table 4

Associations between Actors and Use Cases

Actor	Use Cases
User	UC-01 Log in, UC-02 Log out, UC-03 Change password, UC-06 Search products
Salesperson	UC-05 Manage customers, UC-08 Register sale
Warehouse Clerk	UC-07 Manage products, UC-12 View inventory, UC-13 Adjust stock, UC-15 Register purchase, UC-18 Receive stock alerts
Owner	UC-04 Manage user accounts, UC-09 Void sale, UC-10 Update VAT rate, UC-14 Manage suppliers, UC-16 View sales history, UC-19 Generate reports
None (system)	UC-11 Update stock, UC-17 Check minimum stock, UC-20 Export report to CSV (only reached through «include»)

«include» means that the base use case always performs the included use case (for example, registering a sale always updates the stock, and generating a report always saves it as a CSV file). The «include» arrow goes from the base use case to the included one. Since version 1.3, the diagram has no «extend» relationships: exporting the report to PDF was optional, while the CSV file is always created.

Table 5

«include» Relationships

From (base use case)	Type	To
UC-08 Register sale	«include»	UC-06 Search products
UC-08 Register sale	«include»	UC-11 Update stock

From (base use case)	Type	To
UC-15 Register purchase	«include»	UC-06 Search products
UC-15 Register purchase	«include»	UC-11 Update stock
UC-09 Void sale	«include»	UC-11 Update stock
UC-18 Receive stock alerts	«include»	UC-17 Check minimum stock
UC-19 Generate reports	«include»	UC-20 Export report to CSV

3. Class Diagram

3.1 Overview

A class is the template from which objects are created: it defines their attributes (data) and their methods (actions). The 26 classes of ProvStock are organized in three diagrams to keep them readable:

Diagram A, domain model: the business entities, that is, the "things" that the system stores, such as the sale (`Sale`) or the product (`Product`) (Figures 2 and 3). Since version 1.3, it also includes `ProductInfo` and `SaleFilter`, two small classes that group the data of a product and the criteria to filter the sales.

Diagram B, control classes: one class per Product Backlog item, which keeps the collections of objects and coordinates the use cases; for example, `SalesManager` registers and voids sales (Figure 4).

Diagram C, persistence: the class `FileManager`, which saves the data of the control classes in JSON files and loads it again (Figure 5). The other persistence class, `CsvReportExporter`, is shown in Figure 3 next to the reports that it saves.

The SRS left data access for when the storage mechanism was chosen (SRS, Section 2.4). In version 1.2, CSV files were chosen; in version 1.3 they were replaced by JSON files, one per class, read and written with the Gson library, and all this reading and writing is in `FileManager` (Section 3.4). The user interface is still outside the diagrams.

The code also has support classes that do not appear in the diagrams because they are not part of the domain or the control. In the package view, `ProvStockApp` starts the program, `AppContext` creates the control classes, `MainMenu` shows the options allowed for the role, `ConsoleHelper`, `TextFormat`, and `InputReader` read and print the data, `SampleData` loads the

sample data the first time, and there is one menu class per module (`UserMenu`, `CustomerMenu`, `ProductMenu`, `SalesMenu`, `InventoryMenu`, `PurchaseMenu`, and `ReportMenu`). In the package utils, `IdentificationValidator` validates Ecuadorian identification numbers, and `CsvWriter` writes the rows of a CSV file.

Table 6

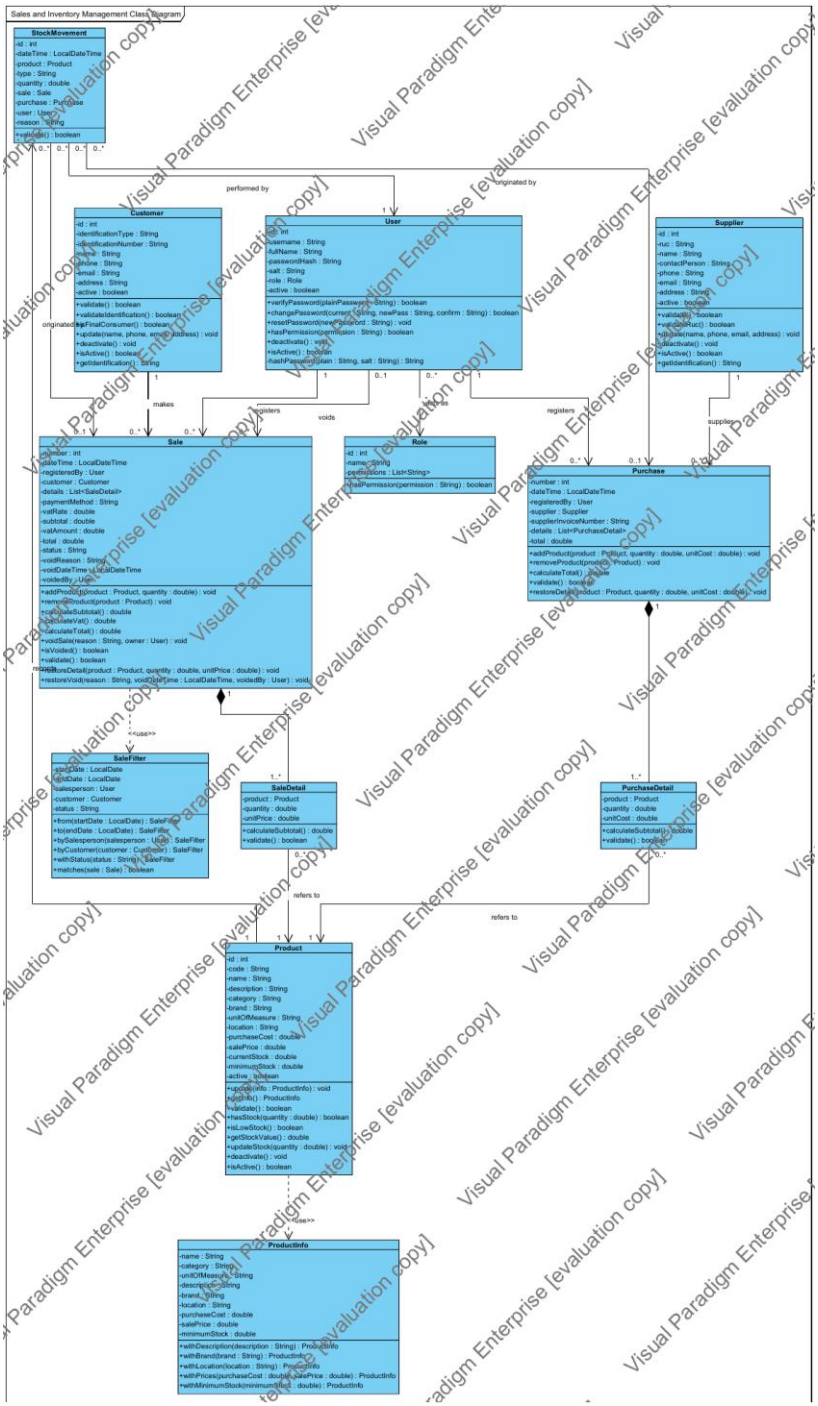
Classes of ProvStock

Class	Diagram	Backlog	Responsibility
Role	A	RF01	One of the three fixed roles and its permissions (Table 3 of the SRS).
User	A	RF01	A person with an account in the system (Table B6).
Customer	A	RF02	A person or organization that buys from the company (Table B1).
Supplier	A	RF06	A company from which the business buys products (Table B3).
Product	A	RF03	An electrical material sold by the company (Table B2).
ProductInfo	A	RF03	The editable data of a product, given to its constructor and to `update()` as one argument object (note N14).
Sale	A	RF04	A sale to a customer registered by a salesperson (Table B4).
SaleFilter	A	RF07	The criteria to filter the sales history: date range, salesperson, customer, and status (note N14).
SaleDetail	A	RF04	One line of a sale.
Purchase	A	RF06	Merchandise bought from a supplier (Table B5).
PurchaseDetail	A	RF06	One line of a purchase.
StockMovement	A	RF05	Any change in the stock of a product (Table B7).
SalesReport	A	RF09	Sales report for a period (FR-09.1).
InventoryReport	A	RF09	Inventory report with the value of the stock (FR-09.2).
PurchaseReport	A	RF09	Purchase report per supplier for a period (FR-09.3).
SlowMovingProductsReport	A	RF09	Products without sales in a period (FR-09.5).
CsvReportExporter	A	RF09	Saves each report as a CSV file in the folder reports (FR-09.4, UC-20); package persistence.
AccessManager	B	RF01	Opens and closes sessions, checks permissions, and manages the accounts (UC-01 to UC-04).
CustomerManager	B	RF02	Registers, updates, searches for, and deactivates customers (UC-05).

Class	Diagram	Backlog	Responsibility
ProductCatalog	B	RF03	Registers, updates, searches for, and deactivates products (UC-06, UC-07).
SalesManager	B	RF04, RF07	Registers and voids sales, keeps the VAT rate, and provides the sales history (UC-08 to UC-10, UC-16).
Inventory	B	RF05	The only class that changes the stock; records every movement (UC-11 to UC-13).
PurchasingManager	B	RF06	Manages suppliers and registers purchases (UC-14, UC-15).
StockMonitor	B	RF08	Calculates the stock alerts (UC-17, UC-18).
ReportGenerator	B	RF09	Creates the four reports from the data read again from the JSON files (UC-19, UC-20).
FileManager	C	Persistence	Saves all the data in JSON files, one per class, and loads it again at startup and before each report (SRS, Section 2.4).

Figure 2

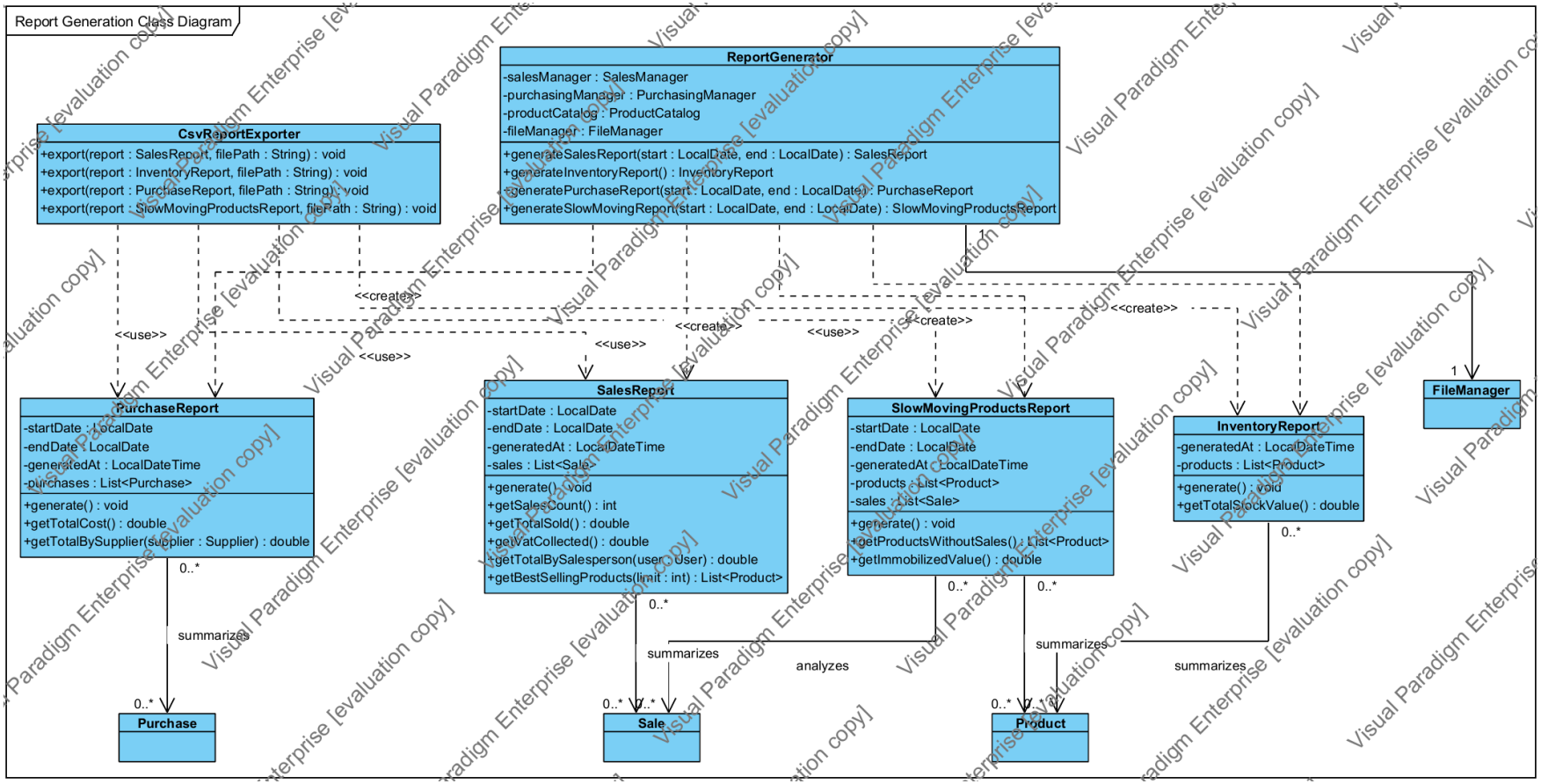
Diagram A: Domain Model (Main Entities)



Note. The multiplicities appear at each end of the associations; the filled diamond indicates composition. In version 1.2, the restore...() methods were added to Sale and Purchase (note N12). In version 1.3, Product receives a `ProductInfo`, and `SaleFilter` selects the sales that match its criteria (note N14).

Figure 3

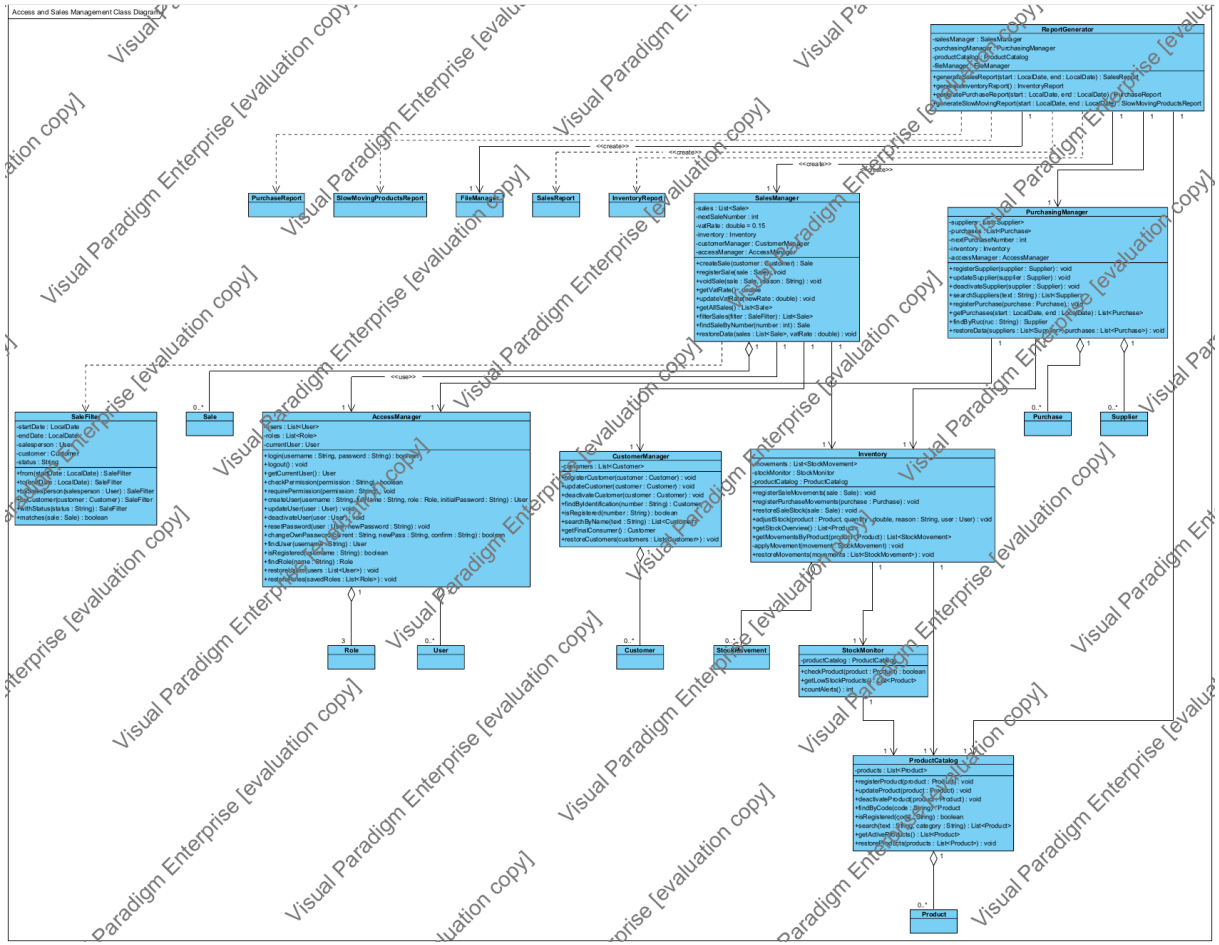
Diagram A: Domain Model (Reports)



Note. Sale, Purchase, Product, and `FileManager` are shown only with their name; their complete definition is in Figures 2 and 5. In version 1.3, `exportToPdf()` was removed from the four reports: the new class `CsvReportExporter` saves them as CSV files («use» dependencies), and `ReportGenerator` keeps a reference to `FileManager` to read the JSON files again before each report.

Figure 4

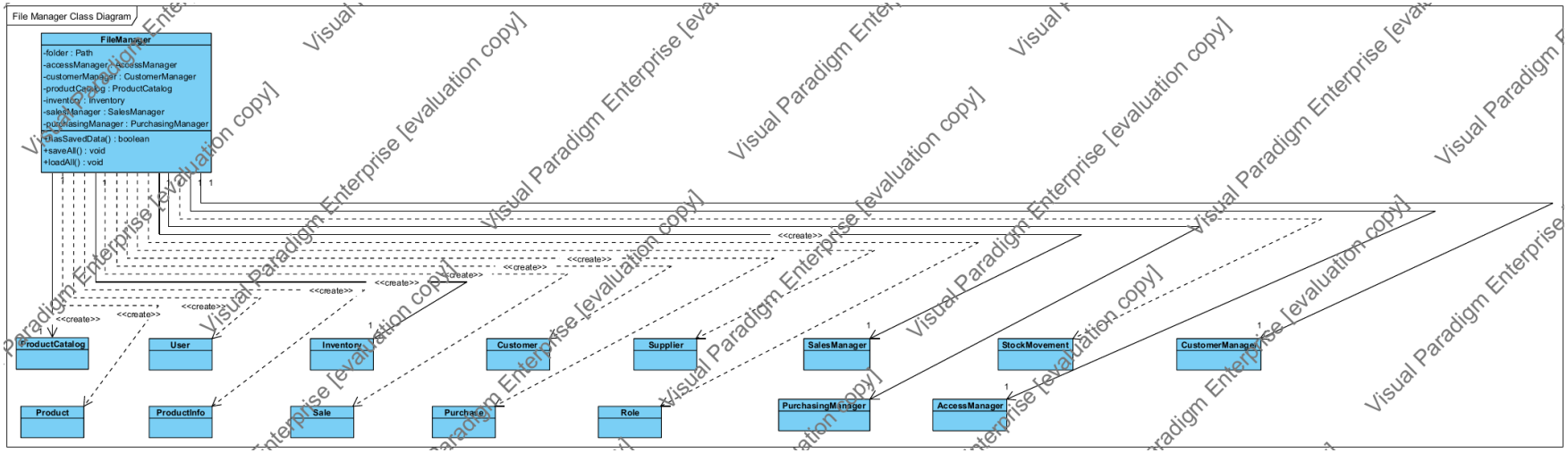
Diagram B: Control Classes (Version 1.3)



Note. The domain classes are shown only with their name. Hollow diamonds indicate aggregation (the control class keeps the collection). Solid lines with an open arrow are directed associations: the source class keeps a reference to the target class (note N8). Dashed arrows indicate the «create» dependencies of ReportGenerator. The restore...() methods are from version 1.2 (note N12). In version 1.3, `ReportGenerator` keeps a reference to `FileManager`, `SalesManager` uses `SaleFilter`, and `AccessManager.requirePermission()` is public.

Figure 5

Diagram C: Persistence (Version 1.3)



Note. The control and domain classes are shown only with their name; their complete definition is in Sections 3.2 and 3.3. Solid lines with an open arrow are directed associations (note N8); the dashed arrows indicate that `FileManager` creates the domain objects (including `ProductInfo` and Role) when it loads the JSON files.

3.2 Domain Classes (Diagram A)

The following boxes show the exact content of each class, ready to be copied into Visual Paradigm. Each box has three parts: the name of the class, its attributes, and its methods. In version 1.2, `Sale` and `Purchase` received the methods `restoreDetail()` and `restoreVoid()`, which only `FileManager` uses when it loads the data (note N12). In version 1.3, Product receives its editable data in a `ProductInfo` object, the method `exportToPdf()` was removed from the four reports, and the new classes `ProductInfo` and `SaleFilter` were added (note N14).

Role — One of the three fixed roles and its permissions (Table 3 of the SRS). *(RF01)*

Role
- id: int - name: String - permissions: List<String>
+ hasPermission(permission: String): boolean

User — A person with an account in the system (Table B6). *(RF01)*

User
- id: int - username: String - fullName: String - passwordHash: String - salt: String - role: Role - active: boolean
+ verifyPassword(plainPassword: String): boolean + changePassword(current: String, newPass: String, confirm: String): boolean + resetPassword(newPassword: String): void + hasPermission(permission: String): boolean + deactivate(): void + isActive(): boolean - hashPassword(plain: String, salt: String): String

Customer — A person or organization that buys from the company (Table B1). *(RF02)*

Customer
- id: int - identificationType: String - identificationNumber: String - name: String - phone: String - email: String - address: String - active: boolean
+ validate(): boolean + validateIdentification(): boolean + isFinalConsumer(): boolean + update(name, phone, email, address): void + deactivate(): void + isActive(): boolean + getIdentification(): String

Supplier — A company from which the business buys products (Table B3). *(RF06)*

Supplier
- id: int - ruc: String - name: String - contactPerson: String - phone: String - email: String - address: String - active: boolean
+ validate(): boolean + validateRuc(): boolean + update(name, phone, email, address): void + deactivate(): void + isActive(): boolean + getIdentification(): String

Product — An electrical material sold by the company (Table B2). *(RF03)*

Product
- id: int - code: String - name: String - description: String - category: String - brand: String - unitOfMeasure: String - location: String - purchaseCost: double - salePrice: double - currentStock: double - minimumStock: double - active: boolean
+ update(info: ProductInfo): void + getInfo(): ProductInfo + validate(): boolean + hasStock(quantity: double): boolean + isLowStock(): boolean + getStockValue(): double + updateStock(quantity: double): void + deactivate(): void + isActive(): boolean

ProductInfo — The editable data of a product (argument object, note N14). *(RF03)*

ProductInfo
- name: String - category: String - unitOfMeasure: String - description: String - brand: String - location: String - purchaseCost: double - salePrice: double - minimumStock: double
+ withDescription(description: String): ProductInfo + withBrand(brand: String): ProductInfo + withLocation(location: String): ProductInfo + withPrices(purchaseCost: double, salePrice: double): ProductInfo + withMinimumStock(minimumStock: double): ProductInfo

Sale — A sale to a customer registered by a salesperson (Table B4). *(RF04)*

Sale
- number: int - dateTime: LocalDateTime - registeredBy: User - customer: Customer - details: List<SaleDetail> - paymentMethod: String - vatRate: double - subtotal: double - vatAmount: double - total: double - status: String - voidReason: String - voidDateTime: LocalDateTime - voidedBy: User
+ addProduct(product: Product, quantity: double): void + removeProduct(product: Product): void + calculateSubtotal(): double + calculateVat(): double + calculateTotal(): double + voidSale(reason: String, owner: User): void + isVoided(): boolean + validate(): boolean + restoreDetail(product: Product, quantity: double, unitPrice: double): void + restoreVoid(reason: String, voidDateTime: LocalDateTime, voidedBy: User): void

SaleDetail — One line of a sale. *(RF04)*

SaleDetail
- product: Product - quantity: double - unitPrice: double
+ calculateSubtotal(): double + validate(): boolean

SaleFilter — The criteria to filter the sales history (argument object, note N14). *(RF07)*

SaleFilter
- startDate: LocalDate - endDate: LocalDate - salesperson: User - customer: Customer - status: String
+ from(startDate: LocalDate): SaleFilter + to(endDate: LocalDate): SaleFilter + bySalesperson(salesperson: User): SaleFilter + byCustomer(customer: Customer): SaleFilter + withStatus(status: String): SaleFilter + matches(sale: Sale): boolean

Purchase — Merchandise bought from a supplier (Table B5). *(RF06)*

Purchase
- number: int - dateTime: LocalDateTime - registeredBy: User - supplier: Supplier - supplierInvoiceNumber: String - details: List<PurchaseDetail> - total: double
+ addProduct(product: Product, quantity: double, unitCost: double): void + removeProduct(product: Product): void + calculateTotal(): double + validate(): boolean + restoreDetail(product: Product, quantity: double, unitCost: double): void

PurchaseDetail — One line of a purchase. *(RF06)*

PurchaseDetail
- product: Product - quantity: double - unitCost: double
+ calculateSubtotal(): double + validate(): boolean

StockMovement — Any change in the stock of a product (Table B7). *(RF05)*

StockMovement
- id: int - dateTime: LocalDateTime - product: Product - type: String - quantity: double - sale: Sale - purchase: Purchase - user: User - reason: String
+ validate(): boolean

SalesReport — Sales report for a period (FR-09.1). *(RF09)*

SalesReport
- startDate: LocalDate - endDate: LocalDate - generatedAt: LocalDateTime - sales: List<Sale>
+ generate(): void + getSalesCount(): int + getTotalSold(): double + getVatCollected(): double + getTotalBySalesperson(user: User): double + getBestSellingProducts(limit: int): List<Product>

InventoryReport — Inventory report with the value of the stock (FR-09.2). *(RF09)*

InventoryReport
- generatedAt: LocalDateTime - products: List<Product>
+ generate(): void + getTotalStockValue(): double

PurchaseReport — Purchase report per supplier for a period (FR-09.3). *(RF09)*

PurchaseReport
- startDate: LocalDate - endDate: LocalDate - generatedAt: LocalDateTime - purchases: List<Purchase>
+ generate(): void + getTotalCost(): double + getTotalBySupplier(supplier: Supplier): double

SlowMovingProductsReport — Products without sales in a period (FR-09.5). *(RF09)*

SlowMovingProductsReport
- startDate: LocalDate - endDate: LocalDate - generatedAt: LocalDateTime - products: List<Product> - sales: List<Sale>
+ generate(): void + getProductsWithoutSales(): List<Product> + getImmobilizedValue(): double

3.3 Control Classes (Diagram B)

Each control class coordinates the use cases of one Product Backlog item and keeps the collections of objects of its module. In addition, it receives in its constructor the other control classes that it needs and keeps them as private attributes (note N8). `Inventory` is the only class that calls `Product.updateStock()` (note N4).

In version 1.1, the following attributes were added to the boxes: `inventory`, `customerManager` and `accessManager` of `SalesManager`; `stockMonitor` and `productCatalog` of `Inventory`; `inventory` and `accessManager` of `PurchasingManager`; `productCatalog` of `StockMonitor`; and `salesManager`, `purchasingManager` and `productCatalog` of `ReportGenerator`. The methods `findUser()`, `findRole()` and `findByRuc()` were also added. In version 1.2, each class that keeps a collection received a `restore...()` method, with which `FileManager` returns the data read from the files (note N12). In version 1.3, the `register...()` methods return void and throw an exception when the data is invalid, the `find...()` methods throw an exception when the object does not exist (note N13), and the methods `isRegistered()`, `restoreRoles()`, and `requirePermission()` were added to the boxes.

AccessManager — Opens and closes sessions, checks permissions, and manages the accounts (UC-01 to UC-04). *(RF01)*

AccessManager
- users: List<User> - roles: List<Role> - currentUser: User
+ login(username: String, password: String): boolean + logout(): void + getCurrentUser(): User + checkPermission(permission: String): boolean + requirePermission(permission: String): void + createUser(username: String, fullName: String, role: Role, initialPassword: String): User + updateUser(user: User): void + deactivateUser(user: User): void + resetPassword(user: User, newPassword: String): void + changeOwnPassword(current: String, newPass: String, confirm: String): boolean + findUser(username: String): User + isRegistered(username: String): boolean + findRole(name: String): Role + restoreUsers(users: List<User>): void + restoreRoles(roles: List<Role>): void

CustomerManager — Registers, updates, searches for, and deactivates customers (UC-05). *(RF02)*

CustomerManager
- customers: List<Customer>
+ registerCustomer(customer: Customer): void + updateCustomer(customer: Customer): void + deactivateCustomer(customer: Customer): void + findByIdentification(number: String): Customer + isRegistered(number: String): boolean + searchByName(text: String): List<Customer> + getFinalConsumer(): Customer + restoreCustomers(customers: List<Customer>): void

ProductCatalog — Registers, updates, searches for, and deactivates products (UC-06, UC-07). *(RF03)*

ProductCatalog
- products: List<Product>
+ registerProduct(product: Product): void + updateProduct(product: Product): void + deactivateProduct(product: Product): void + findByCode(code: String): Product + isRegistered(code: String): boolean + search(text: String, category: String): List<Product> + getActiveProducts(): List<Product> + restoreProducts(products: List<Product>): void

SalesManager — Registers and voids sales, keeps the VAT rate, and provides the sales history (UC-08 to UC-10, UC-16). *(RF04, RF07)*

SalesManager
- sales: List<Sale> - nextSaleNumber: int - vatRate: double = 0.15 - inventory: Inventory - customerManager: CustomerManager - accessManager: AccessManager
+ createSale(customer: Customer): Sale + registerSale(sale: Sale): void + voidSale(sale: Sale, reason: String): void + getVatRate(): double + updateVatRate(newRate: double): void + getAllSales(): List<Sale> + filterSales(filter: SaleFilter): List<Sale> + findSaleByNumber(number: int): Sale + restoreData(sales: List<Sale>, vatRate: double): void

Inventory — The only class that changes the stock; records every movement (UC-11 to UC-13). *(RF05)*

Inventory
- movements: List<StockMovement> - stockMonitor: StockMonitor - productCatalog: ProductCatalog
+ registerSaleMovements(sale: Sale): void + registerPurchaseMovements(purchase: Purchase): void + restoreSaleStock(sale: Sale): void + adjustStock(product: Product, quantity: double, reason: String, user: User): void + getStockOverview(): List<Product> + getMovementsByProduct(product: Product): List<StockMovement> - applyMovement(movement: StockMovement): void + restoreMovements(movements: List<StockMovement>): void

PurchasingManager — Manages suppliers and registers purchases (UC-14, UC-15). *(RF06)*

PurchasingManager
- suppliers: List<Supplier> - purchases: List<Purchase> - nextPurchaseNumber: int - inventory: Inventory - accessManager: AccessManager
+ registerSupplier(supplier: Supplier): void + updateSupplier(supplier: Supplier): void + deactivateSupplier(supplier: Supplier): void + searchSuppliers(text: String): List<Supplier> + registerPurchase(purchase: Purchase): void + getPurchases(start: LocalDate, end: LocalDate): List<Purchase> + findByRuc(ruc: String): Supplier + restoreData(suppliers: List<Supplier>, purchases: List<Purchase>): void

StockMonitor — Calculates the stock alerts (UC-17, UC-18). *(RF08)*

StockMonitor
- productCatalog: ProductCatalog
+ checkProduct(product: Product): boolean + getLowStockProducts(): List<Product> + countAlerts(): int

ReportGenerator — Creates the four reports (UC-19, UC-20). *(RF09)*

ReportGenerator
- salesManager: SalesManager - purchasingManager: PurchasingManager - productCatalog: ProductCatalog - fileManager: FileManager
+ generateSalesReport(start: LocalDate, end: LocalDate): SalesReport + generateInventoryReport(): InventoryReport + generatePurchaseReport(start: LocalDate, end: LocalDate): PurchaseReport + generateSlowMovingReport(start: LocalDate, end: LocalDate): SlowMovingProductsReport

3.4 Persistence Classes (Diagram C)

Since version 1.2, the data is no longer lost when the program is closed. In version 1.3, each class of the model is saved in its own JSON file, written and read with the Gson library (com.google.code.gson, version 2.11.0). JSON keeps the name of every field, so the files can be

opened with any text editor to check the data. All reading and writing of the data is in a single class, `FileManager` (package persistence), which Figure 5 shows with its relationships.

`FileManager` keeps a reference to six control classes. When it saves, it asks them for their collections; when it loads, it creates the domain objects from the files and returns them through their `restore...()` methods (note N12). `ReportGenerator` also calls `loadAll()` before generating a report, so every report is made from the data saved in the files.

FileManager — Saves the data of the control classes in JSON files and loads it again at startup. *(Persistence)*

FileManager
- folder: Path - accessManager: AccessManager - customerManager: CustomerManager - productCatalog: ProductCatalog - inventory: Inventory - salesManager: SalesManager - purchasingManager: PurchasingManager
+ hasSavedData(): boolean + saveAll(): void + loadAll(): void

Internally, `saveAll()` and `loadAll()` call one private method per file (`saveUsers()`, `loadUsers()`, `saveSales()`, `loadSales()`, etc.). Like the other private helper methods, they are not drawn (note N6).

The files are stored in the folder data, inside the project folder. Each file is a JSON array with one object per record:

role.json: the three roles with their list of permissions.

user.json: users with their role, the hash and salt of the password, and whether they are active.

customer.json and supplier.json: customers and suppliers, active and inactive.

product.json: products with all their data except the current stock (note N11).

sale.json and saleDetail.json: each sale with its totals, status, and void data, and its lines with the price of the day; each line keeps its saleNumber.

purchase.json and purchaseDetail.json: each purchase and its lines; each line keeps its purchaseNumber.

stockMovement.json: all the stock movements, in order.

settings.json: the current VAT rate.

The relationships are saved with the key of the other object, like a foreign key: a sale saves the username of the salesperson, the identification number of the customer, and the code of each product. For this reason, the roles, users, customers, products, and suppliers are loaded first; then the sales and purchases; and finally the movements, which rebuild the stock.

Since version 1.3, the reports are saved as CSV files instead of PDF (FR-09.4 of the SRS v1.3). The report classes no longer write files: `CsvReportExporter` (package persistence) receives the report and the path of the file, and `CsvWriter` (package utils) writes the rows with the fields separated by semicolons, so the file opens in columns in Excel. The file name includes the date and time, for example `reports/reporte_ventas_20261026_094846.csv`, so earlier reports are not replaced.

CsvReportExporter — Saves each report as a CSV file in the folder reports (UC-20). *(RF09, package persistence)*

CsvReportExporter

`+ export(report: SalesReport, filePath: String): void`
`+ export(report: InventoryReport, filePath: String): void`
`+ export(report: PurchaseReport, filePath: String): void`
`+ export(report: SlowMovingProductsReport, filePath: String): void`

3.5 Relationships and Multiplicities

In the following tables, each association has the name that appears in the diagram. “Mult. A” is the multiplicity on the side of Class A, and “Mult. B” is the one on the side of Class B. For example, `Customer` 1 — 0..* `Sale` reads: “each sale has exactly one customer, and a customer can have zero or many sales.”

Table 7

Relationships of Diagram A

Class A	Relationship	Class B	Mult. A	Mult. B
User	Association acts as	Role	0..*	1
Customer	Association makes	Sale	1	0..*
User	Association registers	Sale	1	0..*
User	Association voids	Sale	0..1	0..*
Supplier	Association supplies	Purchase	1	0..*
User	Association registers	Purchase	1	0..*
Sale	Composition ◆	SaleDetail	1	1..*
Purchase	Composition ◆	PurchaseDetail	1	1..*
SaleDetail	Association refers to	Product	0..*	1
PurchaseDetail	Association refers to	Product	0..*	1
Product	Association records	StockMovement	1	0..*
Product	Dependency «use»	ProductInfo	—	—
StockMovement	Association performed by	User	0..*	1
StockMovement	Association originated by	Sale	0..*	0..1
StockMovement	Association originated by	Purchase	0..*	0..1
SalesReport	Association summarizes	Sale	0..*	0..*
PurchaseReport	Association summarizes	Purchase	0..*	0..*
InventoryReport	Association summarizes	Product	0..*	0..*
SlowMovingProductsReport	Association summarizes	Product	0..*	0..*
SlowMovingProductsReport	Association analyzes	Sale	0..*	0..*
SaleFilter	Dependency «use»	Sale	—	—
CsvReportExporter	Dependency «use»	SalesReport, InventoryReport, PurchaseReport, SlowMovingProductsReport	—	—

In Diagram B, the eleven «use» dependencies of version 1.0 were changed to directed associations, because the source class keeps the other one as an attribute (note N8). The dependency `AccessManager` «use» `StockMonitor` was removed (note N10). The «create» dependencies of `ReportGenerator` remain. Since version 1.2, the table also includes Diagram C: the associations of `FileManager` with six control classes and its «create» dependency on the

domain classes. In version 1.3, `ReportGenerator` keeps a reference to `FileManager`, `SalesManager` uses `SaleFilter`, and `FileManager` also creates Role and `ProductInfo`.

Table 8

Relationships of Diagrams B and C

Class A	Relationship	Class B	Multiplicity
AccessManager	Aggregation ◇	User	1 — 0..*
AccessManager	Aggregation ◇	Role	1 — 3
CustomerManager	Aggregation ◇	Customer	1 — 0..*
ProductCatalog	Aggregation ◇	Product	1 — 0..*
SalesManager	Aggregation ◇	Sale	1 — 0..*
PurchasingManager	Aggregation ◇	Supplier	1 — 0..*
PurchasingManager	Aggregation ◇	Purchase	1 — 0..*
Inventory	Aggregation ◇	StockMovement	1 — 0..*
SalesManager	Directed association → (inventory)	Inventory	1 → 1
SalesManager	Directed association → (customerManager)	CustomerManager	1 → 1
SalesManager	Directed association → (accessManager)	AccessManager	1 → 1
SalesManager	Dependency «use»	SaleFilter	—
PurchasingManager	Directed association → (inventory)	Inventory	1 → 1
PurchasingManager	Directed association → (accessManager)	AccessManager	1 → 1
Inventory	Directed association → (stockMonitor)	StockMonitor	1 → 1
Inventory	Directed association → (productCatalog)	ProductCatalog	1 → 1
StockMonitor	Directed association → (productCatalog)	ProductCatalog	1 → 1
ReportGenerator	Directed association → (salesManager)	SalesManager	1 → 1
ReportGenerator	Directed association → (purchasingManager)	PurchasingManager	1 → 1
ReportGenerator	Directed association → (productCatalog)	ProductCatalog	1 → 1
ReportGenerator	Directed association → (fileManager)	FileManager	1 → 1
ReportGenerator	Dependency «create»	SalesReport	—
ReportGenerator	Dependency «create»	InventoryReport	—
ReportGenerator	Dependency «create»	PurchaseReport	—
ReportGenerator	Dependency «create»	SlowMovingProductsReport	—
FileManager	Directed association → (accessManager)	AccessManager	1 → 1
FileManager	Directed association → (customerManager)	CustomerManager	1 → 1
FileManager	Directed association → (productCatalog)	ProductCatalog	1 → 1
FileManager	Directed association → (inventory)	Inventory	1 → 1

Class A	Relationship	Class B	Multiplicity
FileManager	Directed association → (salesManager)	SalesManager	1 → 1
FileManager	Directed association → (purchasingManager)	PurchasingManager	1 → 1
FileManager	Dependency «create»	Role, User, Customer, Product, ProductInfo, Supplier, Sale, Purchase, StockMovement	—

Note. The name in parentheses is the attribute that keeps the reference in Class A.

3.6 Design Notes

A UML note is a comment (a rectangle with a folded corner) that clarifies a design decision. The fourteen notes of Table 9 accompany the class diagrams and explain the decisions that cannot be seen in the boxes; they are documented in this table so that the diagrams are not overloaded. N8, N9, and N10 are new in version 1.1, N11 and N12 in version 1.2, and N13 and N14 in version 1.3.

Table 9

UML Notes N1 to N14

Note	Text
N1	Amounts and quantities use double; the amounts of money are rounded to two decimals (BR-06).
N2	The String attributes with fixed values accept only these values: Role.name = SALESPERSON, WAREHOUSE_CLERK, OWNER; Customer.identificationType = CEDULA, RUC, PASSPORT; Product.category = CABLES, LIGHTING, SWITCHES_AND_OUTLETS, PROTECTION_DEVICES, TOOLS, OTHER; Product.unitOfMeasure = UNIT, METER, ROLL, BOX; Sale.paymentMethod = CASH, TRANSFER, CARD; Sale.status = REGISTERED, VOIDED; `StockMovement`.type = SALE, PURCHASE, VOIDED_SALE, ADJUSTMENT.
N3	Customer and Supplier, Sale and Purchase, `SaleDetail` and `PurchaseDetail`, and the four report classes use the same names for the attributes and methods that they share (for example, `validate()`, `calculateTotal()`, and `generate()`), so they are used in the same way.
N4	`Product.updateStock()` must be called only by `Inventory`; the current stock has no public setter (FR-03.3).
N5	A `StockMovement` refers to a sale, to a purchase, or to neither (for adjustments), never to both.
N6	Simple getters and setters are not drawn (for example, `setPaymentMethod()` or `setCustomer()`). Neither are `toString()` nor the private helper methods that only avoid repeating code, such as `checkEditable()`, `checkPeriod()`, or `readDatabase()`. Since version 1.3, `requirePermission()` is public and is drawn.
N7	Official list of permissions for `Role.permissions`, one per row of Table 3 of the SRS: MANAGE_USERS, MANAGE_CUSTOMERS, DEACTIVATE_CUSTOMERS, VIEW_PRODUCTS, MANAGE_PRODUCTS, REGISTER_SALE, VOID_SALE, UPDATE_VAT_RATE, ADJUST_STOCK, VIEW_INVENTORY, MANAGE_SUPPLIERS, REGISTER_PURCHASE, VIEW_SALES_HISTORY, RECEIVE_ALERTS, GENERATE_REPORTS. Table 10 shows which permissions each role has.

Note	Text
N8	Each control class receives in its constructor the control classes that it uses and keeps them as private attributes; for example, `SalesManager` keeps `inventory`, `customerManager` and `accessManager`. For this reason, they are drawn as directed associations and not as «use» dependencies, which, according to Table 2, are not kept as attributes.
N9	The fixed values of N2 and the permissions of N7 are declared as public static final constants in their class (for example, Role.OWNER or Sale.VOIDED). The constants are not drawn.
N10	The check of the minimum stock at login (FR-08.1) is done by the user interface right after `AccessManager.login()`, by calling `StockMonitor.countAlerts()`. For this reason, `AccessManager` does not depend on `StockMonitor`.
N11	The current stock is not saved in `product.json`. When loading, `Inventory.restoreMovements()` applies again, in order, all the saved movements, so the stock is always the sum of its movements (FR-03.3, N4).
N12	The `restore...()` methods of the control classes, of `Sale`, and of `Purchase` are used only by `FileManager` when loading. They do not validate the business rules again (stock, active products, permissions), because that data was already validated when it was registered; for example, `Sale.restoreDetail()` keeps the price of the day of the sale.
N13	Exceptions instead of special values (Martin, 2008, pp. 46, 104, 110): the `find...()` methods throw `NoSuchElementException` when the object does not exist instead of returning null, and the `register...()` methods return void and throw `IllegalArgumentException` when the data is invalid or the key already exists, instead of returning false. To check before registering, the control classes offer `isRegistered()`. The menus show the message of the exception as “Error: ...”.
N14	Argument objects (Martin, 2008, p. 43): Product receives its editable data in a `ProductInfo` object (constructor and `update()`) instead of nine parameters, and `SalesManager.filterSales()` receives a `SaleFilter` instead of five parameters, several of them null to mean “no filter”. Their methods `with...()`, `from()`, `to()`, and `by...()` return the same object, so the optional values can be chained; for example, new `SaleFilter().from(start).to(end)`.

Table 10 shows the permissions that `AccessManager` assigns to each role when the system starts. They come from the associations of Table 4 and from the actor generalization of Table 3: the owner inherits the permissions of the salesperson and the warehouse clerk. Before delivery, the team must confirm that it matches Table 3 of the SRS, especially DEACTIVATE_CUSTOMERS, which was left only for the owner.

Table 10

Permissions Assigned to Each Role

Permission	Use case	Salesperson	Warehouse Clerk	Owner
MANAGE_USERS	UC-04	—	—	✓
MANAGE_CUSTOMERS	UC-05	✓	—	✓
DEACTIVATE_CUSTOMERS	UC-05	—	—	✓
VIEW_PRODUCTS	UC-06	✓	✓	✓

Permission	Use case	Salesperson	Warehouse Clerk	Owner
MANAGE_PRODUCTS	UC-07	—	✓	✓
REGISTER_SALE	UC-08	✓	—	✓
VOID_SALE	UC-09	—	—	✓
UPDATE_VAT_RATE	UC-10	—	—	✓
VIEW_INVENTORY	UC-12	—	✓	✓
ADJUST_STOCK	UC-13	—	✓	✓
MANAGE_SUPPLIERS	UC-14	—	—	✓
REGISTER_PURCHASE	UC-15	—	✓	✓
VIEW_SALES_HISTORY	UC-16	—	—	✓
RECEIVE_ALERTS	UC-18	—	✓	✓
GENERATE_REPORTS	UC-19, UC-20	—	—	✓

Note. ✓ = the role has the permission; — = it does not. UC-03 (change password) and UC-02 (log out) need no permission because any user with an open session can use them.

3.7 Rules Implemented by the Methods

The following rules come from the SRS and must be respected when the methods are programmed. The rows marked (v1.1), (v1.2), or (v1.3) describe behaviors that the code already has and that were not written in the previous versions.

Table 11

Rules Implemented by the Methods

Method	Rule
`Sale.addProduct()`	Rejects inactive products and quantities greater than the stock; if the product is already in the sale, the quantities are added (FR-03.5, BR-03).
`Sale` / `Purchase`: `addProduct()`, `removeProduct()`	Fail if the sale or purchase already has a number, because registered sales and purchases cannot be edited (BR-04).
`Purchase.addProduct()`, `Purchase.validate()`	Reject inactive products and suppliers (FR-03.5, Table B5).
`Purchase.addProduct()`, `PurchaseDetail.validate()` (v1.2)	The unit cost must be greater than 0 (BR-02).
`Purchase.validate()` (v1.2)	The supplier invoice number is optional (UC-15): a purchase without an invoice number is valid and is saved with that field empty.
`StockMonitor.getLowStockProducts()`	Considers only the active products.
`Customer.isFinalConsumer()`	The “Final Consumer” uses the identification number 9999999999999 and cannot be modified or deactivated (FR-02.6).

Method	Rule
`AccessManager.createUser()`, `resetPassword()`	Validate that the password has at least 8 characters (NFR-02).
Methods `update()`	Do not change the keys (`code`, `identificationNumber`, `ruc`, `username`), so BR-08 is also met after an update.
Sensitive operations	`voidSale`, `updateVatRate`, user management, and `adjustStock` check the permission of the user again, in addition to the main menu (NFR-02, UI-02).
Initial stock	A new product starts with stock 0. The initial stock is loaded with `Inventory.adjustStock()` and the reason "Initial inventory" (AS-05, FR-05.4).
`Product.isLowStock()`, `StockMonitor.checkProduct()` (v1.1)	There is an alert when the product is active and its `currentStock` is less than or equal to its `minimumStock`.
`Product.updateStock()` (v1.1)	Rejects a movement that would leave the stock negative.
`Sale.number`, `Purchase.number` (v1.1)	While the number is 0, the sale or purchase is not registered yet and can be edited. `SalesManager.registerSale()` and `PurchasingManager.registerPurchase()` assign the number and the date and time when it is registered.
`Inventory.registerSaleMovements()` (v1.1)	Checks the stock of all the lines first and only then applies the movements, so that a sale is never half deducted.
`Sale.voidSale()` (v1.1)	Voids only registered sales that are not voided and requires the reason; it saves who voided the sale and when (BR-05, FR-04.8).
`SalesManager.updateVatRate()` (v1.1)	Accepts rates between 0 and 0.99. The new rate applies only to the sales created afterward, because each `Sale` keeps its own `vatRate`.
`User.hashPassword()` (v1.1)	The password is saved as a SHA-256 hash with a random `salt`, never in plain text. The constructor of `User` receives the initial password.
`AccessManager.deactivateUser()` (v1.1)	A user cannot deactivate their own account.
Constructor of `AccessManager` (v1.1)	Creates the three roles with the permissions of Table 10 and an initial owner account, so that the system can be accessed the first time.
Constructor of `CustomerManager` (v1.1)	Creates the Final Consumer with identification type RUC and number 9999999999999.
`Customer.validateIdentification()`, `Supplier.validateRuc()` (v1.1)	The cédula is validated with its province and its check digit (modulo 10); the RUC, with its province, its third digit, and its ending; the passport accepts 5 to 20 letters or digits.
`Supplier.contactPerson` (v1.1)	It is changed with its setter, because `update(name, phone, email, address)` does not include it.
`FileManager.saveAll()` (v1.2)	It is called after every menu operation, even if the operation fails halfway, so that the files always contain what is in memory. Each file is first written to a temporary file that then replaces the previous one, so that it is never left half written.
`FileManager.loadAll()` (v1.2)	Loads in order: roles, users, customers, products, and suppliers; then sales and purchases with their lines; finally the movements (N11). If a file has an error, the program shows the file and the problem and stops without overwriting the data.
First run (v1.2)	If the folder `data` does not exist, the sample data is loaded only once and saved. The following times, the files are loaded.
`SalesManager.restoreData()`, `PurchasingManager.restoreData()` (v1.2)	The next sale or purchase number continues from the highest saved number.

Method	Rule
`AccessManager.restoreUsers()` (v1.2)	If the file has no users, the initial owner account is kept. The passwords are not hashed again: the hash and the salt are loaded as they were saved.
`CustomerManager.restoreCustomers()` (v1.2)	If the file does not contain the Final Consumer, it is created again (FR-02.6).
Texts in JSON (v1.3)	Gson escapes the special characters, so names and addresses are saved exactly as they were written (the rule of version 1.2 that changed semicolons into commas no longer applies).
`find...()` methods (v1.3)	Throw `NoSuchElementException` with a message such as “No existe el producto P001.” when the object does not exist; they never return null (N13).
`register...()` methods (v1.3)	Return void. They throw `IllegalArgumentException` if the data is not valid or the key is already registered, and `SecurityException` if the user does not have the permission (N13).
`AccessManager.requirePermission()` (v1.3)	Public; throws `SecurityException` if the logged-in user does not have the permission. The control classes call it instead of repeating the check.
`AccessManager.restoreRoles()` (v1.3)	If `role.json` does not exist yet, the three fixed roles created by the constructor are kept.
`ReportGenerator`.generate...`Report()` (v1.3)	Calls `FileManager.loadAll()` first, so the report is made from the data saved in the JSON files.
`CsvReportExporter.export()` (v1.3)	Writes the report in the folder reports, with a file name that includes the date and time, and with the amounts with two decimals.
`SaleFilter.matches()` (v1.3)	A criterion that was not given (null) is not applied; with no criteria, every sale matches.

4. Traceability

Traceability makes it possible to follow each requirement to the design that solves it. Table 12 relates each Product Backlog item to the use cases and the classes that implement it. Every requirement of the SRS v1.3 is covered by at least one method, and every class and method is justified by a requirement.

Table 12

Traceability between Requirements, Use Cases, and Classes

Backlog	Use Cases	Control Class	Domain Classes
RF01	UC-01 to UC-04	AccessManager	User, Role
RF02	UC-05	CustomerManager	Customer
RF03	UC-06, UC-07	ProductCatalog	Product, ProductInfo
RF04	UC-08 to UC-10	SalesManager	Sale, SaleDetail
RF05	UC-11 to UC-13	Inventory	StockMovement, Product
RF06	UC-14, UC-15	PurchasingManager	Supplier, Purchase, PurchaseDetail
RF07	UC-16	SalesManager	Sale, SaleFilter
RF08	UC-17, UC-18	StockMonitor	Product
RF09	UC-19, UC-20	ReportGenerator	SalesReport, InventoryReport, PurchaseReport, SlowMovingProductsReport, CsvReportExporter
Persistence (SRS, Section 2.4)	All	FileManager	Role, User, Customer, Product, ProductInfo, Supplier, Sale, Purchase, StockMovement

5. Changes from Previous Versions

5.1 Changes from the Diagram of Homework 07

Table 13 summarizes the differences between version 1.0 and the class diagram of homework 07, and the reason for each change.

Table 13

Changes from Homework 07

Change	Reason
Class removed: `Date`	`java.util.Date` is deprecated; `LocalDate` and `LocalDateTime` are used, and FR-04.1 requires the time of the sale.
`Inventory` no longer stores `currentStock`	The stock was stored twice (in `Product` and in `Inventory`). Now it is stored only in `Product` and changes only through stock movements (FR-03.3). `Inventory` became the control class that applies the movements.
New class `StockMovement`	FR-05.6 and Table B7 require every stock movement to be recorded.
Class removed: `Alert`	The alert is calculated from the stock and the minimum stock of each product and disappears automatically (FR-08.4); it is calculated by `StockMonitor`.
`SalesHistory` was merged into `SalesManager`	Both work on the same list of sales.
`Report` was split into four classes	Each report has different data (FR-09.1 to FR-09.5).
Replaced `Role.description` and `Role.getPermissions()`	The roles are fixed (FR-01.1); `hasPermission()` checks a permission of Table 3.
Quantities and stock as `double`	Cables are sold by the meter (note of Table B2).
New attributes and methods	`Product.location` (CH-01), `Sale.vatRate` and the void data (BR-05, FR-04.8), `User.resetPassword()` (FR-01.7), `validate()` in `Customer` and `Supplier` (FR-02.2), `removeProduct()` in `Purchase`.
Control classes added (Diagram B)	One class per Product Backlog item to keep the collections and coordinate the use cases.

5.2 Changes in Version 1.1

Table 14 summarizes what changed between version 1.0 and version 1.1 of this document. All the changes come from the Java implementation: the goal is that the diagram shows what the code really does.

Table 14

Changes in Version 1.1 from Version 1.0

Change	Reason
The eleven «use» dependencies between control classes become directed associations, and their attributes are added to the boxes of Section 3.3 (Table 8, Figure 4)	The control classes use the others in several methods and need to keep the reference. For example, `StockMonitor.getLowStockProducts()` receives no parameters and needs the product catalog. According to Table 2, a dependency is not kept as an attribute, so the previous diagram could not be programmed as it was (note N8).
Dependency removed: `AccessManager` «use» `StockMonitor`	The check of the minimum stock at login is done by the interface after `login()`; `AccessManager` does not use `StockMonitor` (note N10).
New methods `AccessManager.findUser()`, `AccessManager.findRole()` and `PurchasingManager.findByRuc()`	User management needs to find an account by its username and a role by its name; the registration of purchases needs to find the supplier by its RUC.
New notes N8, N9, and N10, and extended N6 (Table 9)	They explain the directed associations, the constants that replace the `enum` for now, and the change in the stock check at login. N6 now clarifies that `toString()` and the private helper methods are not drawn either.
New Table 10, permissions per role	It records which permissions each role has in the code; before, the document only referred to Table 3 of the SRS.
New rules in Table 11 (marked v1.1)	They describe behaviors that the code already applies: low stock, number 0 for unregistered sales and purchases, VAT range, hashed passwords, Final Consumer, and validation of identification numbers.
“Directed association (→)” added to Table 2	It is the new notation used in Figure 4.
Tables 10 to 15 were renumbered and the section of changes was divided into two subsections	Because of the new table of permissions and the new table of changes.
The domain classes, Figures 1 to 3, and Tables 3 to 7 do not change	—

5.3 Changes in Version 1.2

Table 15 summarizes what changed between version 1.1 and version 1.2. The main change is persistence: before, everything that was registered was lost when the program was closed.

Table 15

Changes in Version 1.2 from Version 1.1

Change	Reason
New class `FileManager` in the package `persistence`, new Section 3.4, and new Figure 5 (Diagram C)	The data lived only in memory. The SRS left data access for when the storage was chosen (Section 2.4); CSV files were chosen because they need no external libraries.
New methods `restore...()` in the six control classes that keep collections (Figure 4)	`FileManager` needs to return the objects read to them without going through the registration validations (note N12).
New methods `Sale.restoreDetail()`, `Sale.restoreVoid()` and `Purchase.restoreDetail()` (Figure 2)	`addProduct()` uses the current price and `voidSale()` the current time; a saved sale must keep the price of the day and the original void date.
New notes N11 and N12 (Table 9)	They explain that the stock is rebuilt from the movements and when these methods are used: `restore...()`.
Updated use case diagram (Figure 1, Table 5, note of Figure 1, and note N10)	Review of the diagram against the course slides: Manage customers «extend» Register sale was removed (registering a customer during a sale is alternative flow 2a of UC-08), and so were the «include» relationships that described how the system works internally (Log in and Update stock toward Check minimum stock, Adjust stock toward Update stock). Receive stock alerts now includes Check minimum stock, and the generalizations toward User share one triangle. The code does not change.
New purchase rules in Table 11 (v1.2)	The code validates that the unit cost is greater than 0 (BR-02) and makes the supplier invoice number optional, as UC-15 states in the Use Case Specifications.
New rules (v1.2) in Table 11	They describe the loading order, saving after every operation, the first run, and the numbering after loading.
`FileManager` in Tables 6, 8, 12, and 13; Sections 3.4 to 3.6 become 3.5 to 3.7	Because of the new class and the new diagram.
New methods that are not drawn (N6): `getAllCustomers()`, `getAllProducts()`, `getAllSuppliers()`, `getAllMovements()`, `User.getPasswordHash()` and `User.getSalt()`, and restoration constructors in `User` and `StockMovement`	They are getters and constructors that `FileManager` needs to read all the data, including inactive records, and to create the objects with their original date and password.

5.4 Changes in Version 1.3

Table 16 summarizes what changed between version 1.2 and version 1.3. The changes come from two sources: the requests of the instructor (JSON database and CSV reports) and the review of the code with the book Clean Code (Martin, 2008).

Table 16

Changes in Version 1.3 from Version 1.2

Change	Reason
Data saved in JSON files, one per class (`role.json`, `user.json`, `customer.json`, `product.json`, `supplier.json`, `sale.json`, `saleDetail.json`, `purchase.json`, `purchaseDetail.json`, `stockMovement.json`, and `settings.json`), with the Gson library (Section 3.4, note N11, Table 11)	The instructor asked for a JSON database with one file per class. JSON keeps the name of each field, and Gson writes and reads the files without code to separate the fields by hand.
`exportToPdf()` removed from the four report classes; new class `CsvReportExporter` in the package persistence (Figure 3, Tables 6 to 8)	The instructor asked to export the reports to CSV (FR-09.4 of the SRS v1.3). A report should only calculate its data; writing files is a different responsibility (Single Responsibility Principle, Martin, 2008, p. 138).
`ReportGenerator` keeps a reference to `FileManager` (Figures 3 and 4, Table 8)	The reports read the JSON files again before they are generated, as the instructor asked.
Use case diagram: UC-19 Generate reports includes UC-20 Export report to CSV (Figure 1, Tables 4 and 5)	The report is always saved as a CSV file after it is displayed; before, exporting it to PDF was optional («extend»).
New class `ProductInfo`; the constructor of Product and `Product.update()` receive a `ProductInfo`; new method `getInfo()` (Figure 2, note N14)	The constructor had eleven parameters and `update()` nine. Clean Code recommends grouping them in an argument object (Martin, 2008, pp. 40, 43).
New class `SaleFilter`; `SalesManager.filterSales(filter: SaleFilter)` (Figures 2 and 4, note N14)	`filterSales()` had five parameters, and the caller passed null for the filters that were not used (Martin, 2008, p. 43).
The `find...()` methods throw `NoSuchElementException`; the `register...()` methods return void and throw an exception; new `isRegistered()` methods (Figure 4, note N13, Table 11)	Returning null or false forces every caller to check the result and is easy to forget (Martin, 2008: “Prefer Exceptions to Returning Error Codes”, p. 46, and “Don't Return Null”, p. 110).
`AccessManager.requirePermission()` becomes public; new `AccessManager.restoreRoles()` (Figure 4)	The same permission check was repeated in several classes (G5 Duplication, Martin, 2008, p. 289). The roles are now saved in `role.json`.
The package view was divided into one menu class per module, and `CsvWriter` replaces `SimplePdfWriter` (Section 3.1)	`ProvStockApp` had more than 1,000 lines and all the menus (Single Responsibility Principle). These support classes are not drawn.
New notes N13 and N14; N6 and N11 updated; new rules (v1.3) in Table 11; Tables 6 to 8 and 12 updated	Because of the changes above. The 103 unit tests do not change: 67 pass and 36 fail on purpose, as the instructor asked.
Section of planned future changes to the class diagram removed, with its table; Sections 5 and 6 renumbered as 4 and 5; notes N1, N2, N3, and N9 rewritten	The document describes only the classes of the current program, so the plans for a later class diagram were taken out.